TECNICATURA UNIVERSITARIA EN AUTOMATIZACIÓN Y ROBÓTICA · PROYECTO INTEGRADOR

Documentación Técnica Integral y Código en C: Radar Ultrasónico Pendular con ESP32

Asignaturas: Int. a los Sistemas Robóticos · Electrónica Analógica · Programación en C

Entrega Final: Semana 16/11/2026

1. Arquitectura Sistémica y Mapa de Hardware (Sistemas Robóticos)

El sistema robótico proyectado consiste en un radar de barrido pendular gobernado por un microcontrolador ESP32. El prototipo articula seis subsistemas acoplados: la unidad de control (ESP32 operando a 3,3 V), la etapa de electrónica de potencia (puente H discreto alimentado con +12 V), el actuador electromecánico (motor de corriente continua con reducción mecánica), el mecanismo pendular encargado de desplazar angularmente el cabezal entre 0° y 180° en pasos discretos de 15°, el subsistema de detección ultrasónica (sensor PING/HC-SR04) y el panel de indicación visual compuesto por tres LEDs de estado.

Para garantizar mediciones libres de ruido mecánico y rebotes acústicos, el control ejecuta una secuencia de movimiento paso a paso: acciona el motor durante un intervalo calibrado (120 ms), aplica frenado dinámico en el puente H, adquiere el eco ultrasónico con el péndulo estático y actualiza los indicadores luminosos antes de decidir el sentido del siguiente paso.

Subsistema	Macro en Código (setup.h)	Pin ESP32	Dirección y Nivel Eléctrico	Función en el Prototipo
Puente H (Rama Izq.)	`PIN_MOTOR_IN1`	GPIO 18	Salida Digital (0 V / 3,3 V)	Dispara el driver BS170 izquierdo (avance horario).
Puente H (Rama Der.)	`PIN_MOTOR_IN2`	GPIO 19	Salida Digital (0 V / 3,3 V)	Dispara el driver BS170 derecho (retroceso antihorario).
Sensor Ultrasónico	`PIN_SENSOR_PING`	GPIO 4	Bidireccional (Divisor 5 V → 3,3 V)	Emite pulso de disparo (10 µs) y mide ancho de eco en µs.
Indicador Lejano / Libre	`PIN_LED_VERDE`	GPIO 21	Salida Digital (Resistor 330 Ω)	Indica área despejada o detección mayor a 40,0 cm.
Indicador Precaución	`PIN_LED_AMARILLO`	GPIO 22	Salida Digital (Resistor 330 Ω)	Indica objeto a distancia media (entre 15,0 cm y 40,0 cm).

Subsistema	Macro en Código (setup.h)	Pin ESP32	Dirección y Nivel Eléctrico	Función en el Prototipo
Indicador Alerta Cercana	`PIN_LED_ROJO`	GPIO 23	Salida Digital (Resistor 330 Ω)	Alerta de objeto cercano a menos de 15,0 cm del sensor.

2. Diseño y Cálculos del Puente H Discreto (Electrónica Analógica)

Para invertir el giro del motor de CC sin circuitos integrados comerciales, se diseñó un puente H simétrico con MOSFETs de potencia en encapsulado TO-220. Se descartó el uso de transistores BJT de baja señal (BC327/BC337, limitados a 800 mA y 625 mW) como llaves principales debido a que la corriente de arranque e inversión del motor supera 1 A. La etapa de potencia emplea dos MOSFETs Canal P (**IRF9510**, 4,0 A) en el lado alto (High-Side) y dos MOSFETs Canal N (**IRF510**, 5,6 A) en el lado bajo (Low-Side), comandados por dos MOSFETs Canal N **BS170** que adaptan los 3,3 V del ESP32 a la tensión de fuente de +12 V.

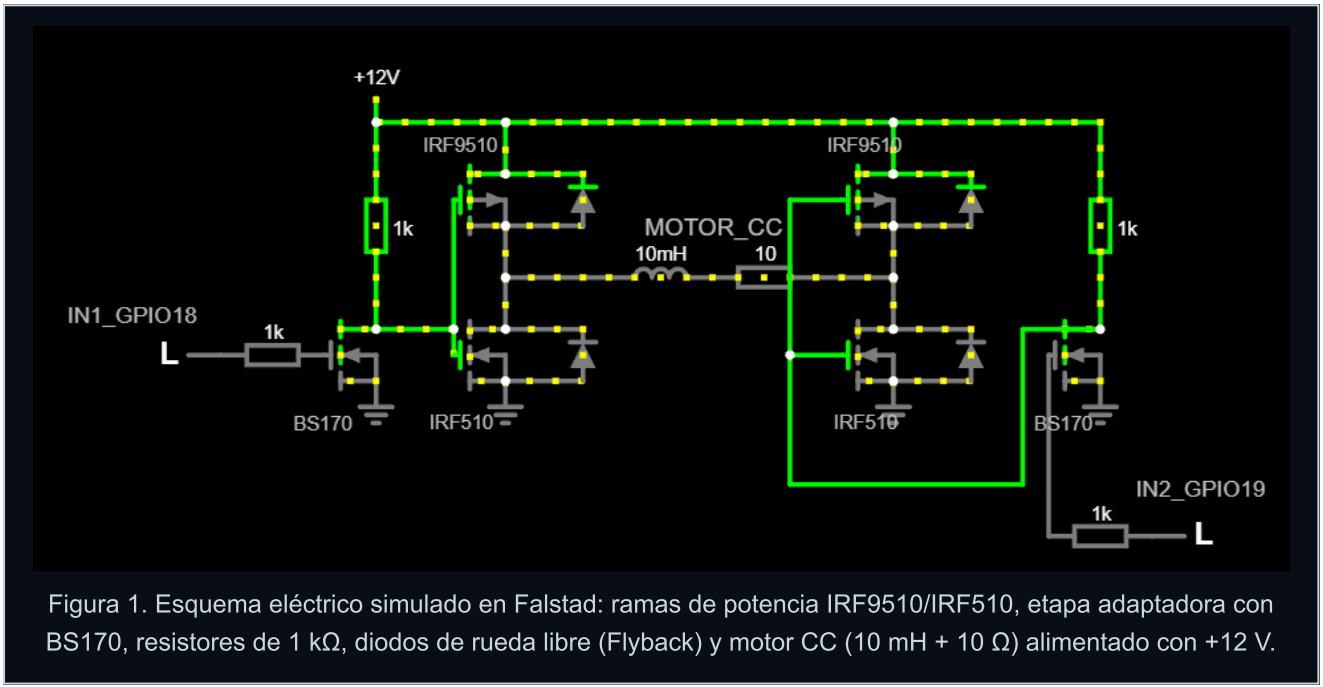


Figura 1. Esquema eléctrico simulado en Falstad: ramas de potencia IRF9510/IRF510, etapa adaptadora con BS170, resistores de 1 kΩ, diodos de rueda libre (Flyback) y motor CC (10 mH + 10 Ω) alimentado con +12 V.

2.1. Cálculo de Corriente de Régimen y Disipación Térmica

En régimen permanente, la inductancia de armadura (L_m = 10 mH) se comporta como un cortocircuito en continua, quedando la corriente limitada por la resistencia del bobinado (R_m = 10 Ω) y las resistencias de conducción $R_{DS(on)}$ de los dos MOSFETs activos en diagonal ($R_{DS(on,\ P)} \approx 0{,}60$ Ω en el IRF9510 y $R_{DS(on,\ N)} \approx 0{,}54$ Ω en el IRF510):

$$I_{motor} = V_{CC} \div (R_m + R_{DS(on,\,P)} + R_{DS(on,\,N)}) = 12\ V \div (10\ \Omega + 0{,}60\ \Omega + 0{,}54\ \Omega) = 12\ V \div 11{,}14\ \Omega \approx 1{,}08\ A$$

$$V_{motor} = I_{motor} \times R_m = 1{,}08\ A \times 10\ \Omega = 10{,}80\ V \quad | \quad P_{D(IRF9510)} = I_{motor}^2 \times R_{DS(on,\,P)} = (1{,}08\ A)^2 \times 0{,}60\ \Omega \approx 0{,}70\ W$$

• **Caída en las llaves:** $V_{DS(P)}$ = 1,08 A × 0,60 Ω = 0,65 V y $V_{DS(N)}$ = 1,08 A × 0,54 Ω = 0,58 V.
• **Conmutación de compuertas (3,3 V → 12 V):** Cuando el GPIO entrega 3,3 V (nivel alto H), supera la tensión umbral del driver BS170 ($V_{GS(th)} \approx 2{,}1$ V), saturándolo y llevando el nodo de compuertas a 0 V (I_{driver} = 12 V ÷ 1 kΩ = 12 mA). Con el nodo en 0 V, el IRF9510 superior recibe V_{GS} = −12 V (**Conduce**) y el IRF510 inferior recibe V_{GS} = 0 V (**Corte**). Cuando el GPIO entrega 0 V (nivel bajo L), el BS170 entra en corte y el resistor pull-up de 1 kΩ eleva el nodo a +12 V, apagando el IRF9510 (V_{GS} = 0 V) y encendiendo el IRF510 inferior (V_{GS} = +12 V).

Estado del Motor	GPIO 18 (IN1)	GPIO 19 (IN2)	Rama Izquierda (Q1 P / Q2 N)	Rama Derecha (Q3 P / Q4 N)	Respuesta Eléctrica en Bornes
MOTOR_DETENIDO	L (0,0 V)	L (0,0 V)	Q1 **Corte** · Q2 **Sat.** (0 V)	Q3 **Corte** · Q4 **Sat.** (0 V)	Frenado dinámico a GND (0,0 A)
SENTIDO_HORARIO	H (3,3 V)	L (0,0 V)	Q1 **Sat.** (+12 V) · Q2 **Corte**	Q3 **Corte** · Q4 **Sat.** (0 V)	Giro Horario (+10,8 V · 1,08 A →)
SENTIDO_ANTIHORARIO	L (0,0 V)	H (3,3 V)	Q1 **Corte** · Q2 **Sat.** (0 V)	Q3 **Sat.** (+12 V) · Q4 **Corte**	Giro Antihorario (−10,8 V · ← 1,08 A)

2.2. Acondicionamiento del Sensor PING y Guía de Pinout en Laboratorio

El sensor ultrasónico opera con alimentación de 5,0 V, por lo que su pulso de retorno (Echo/SIG) alcanza 5,0 V y no debe conectarse de forma directa al GPIO 4 del ESP32 (cuyo límite máximo es 3,6 V). Se intercala un divisor resistivo formado por R_1 = 1 kΩ en serie y R_2 = 2,2 kΩ a masa, obteniendo una tensión segura en el pin del microcontrolador: V_{GPIO4} = 5,0 V × 2,2 kΩ ÷ (1,0 kΩ + 2,2 kΩ) = 3,44 V (reconocido como nivel lógico alto sin riesgo de daño).

Componente	Encapsulado	Distribución de Terminales (Vista frontal con inscripción hacia el observador)
IRF9510 / IRF510	TO-220	**Pata 1 (Izq.):** Gate (Compuerta) · **Pata 2 (Centro):** Drain (Al motor) · **Pata 3 (Der.):** Source (+12 V o GND)
BS170 (Driver)	TO-92	**Pata 1 (Izq.):** Drain (Al pull-up 1 kΩ) · **Pata 2 (Centro):** Gate (A R 1 kΩ del GPIO) · **Pata 3 (Der.):** Source (GND)
BC327 / BC337	TO-92	**Pata 1 (Izq.):** Colector · **Pata 2 (Centro):** Base (Con resistor limitador) · **Pata 3 (Der.):** Emisor

Componente	Encapsulado	Distribución de Terminales (Vista frontal con inscripción hacia el observador)
1N4007 / 1N5819	DO-41	**Banda gris (Cátodo):** Apunta siempre hacia el potencial superior (+12 V arriba, o bornes del motor abajo).

3. Algoritmo y Máquina de Estados Finitos (Programación en C)

El software embebido se estructuró sobre el framework ESP-IDF aplicando programación modular en lenguaje C. El flujo principal en `main.c` implementa una Máquina de Estados Finitos (FSM) determinista con cuatro estados (`ESTADO_INICIALIZACION`, `ESTADO_BARRIDO_HORARIO`, `ESTADO_LECTURA` y `ESTADO_BARRIDO_ANTIHORARIO`), coordinando los módulos independientes `setup`, `motor` y `sensor`.

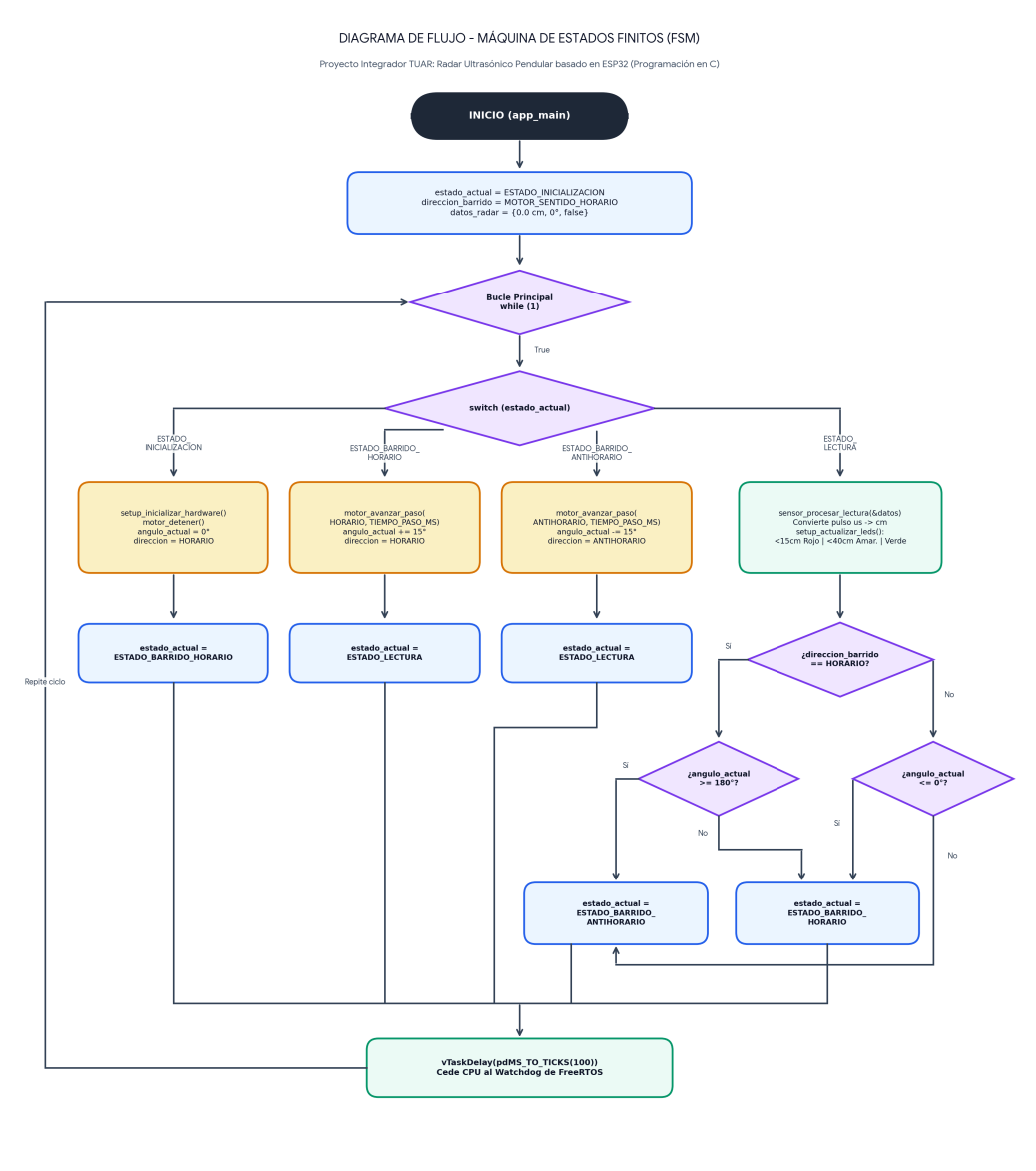


Figura 2. Diagrama de flujo del bucle principal en C: transición entre inicialización, avance por pasos de 15°, lectura ultrasónica, encendido de LEDs e inversión automática en los límites de 0° y 180°.

4. Documentación del Código Fuente Modular en C

A continuación se presenta el código completo y documentado de los siete archivos que conforman el repositorio del proyecto (`CMakeLists.txt`, `setup.h`, `setup.c`, `motor.h`, `motor.c`, `sensor.h`, `sensor.c` y `main.c`), listo para compilar en ESP-IDF y justificar línea por línea durante la defensa oral.

4.1. Configuración de Compilación (main/CMakeLists.txt) y Cabecera de Hardware (setup.h)

El archivo `setup.h` emplea directivas `#define` para asociar los pines físicos sin consumir memoria RAM y declara el tipo enumerado `estado_sistema_t` que gobierna la máquina de estados. Las constantes de umbral llevan el sufijo `f` (`float` de 32 bits) para aprovechar la Unidad de Punto Flotante (FPU) en hardware del ESP32, evitando la promoción a `double` (64 bits) por software.

```
// Archivo: main/CMakeLists.txt
idf_component_register(SRCS "main.c" "motor.c" "sensor.c" "setup.c"
                    INCLUDE_DIRS "."
                    PRIV_REQUIRES esp_driver_gpio esp_timer)
```

```
// Archivo: main/setup.h — Mapa de pines del ESP32 (#define), constantes y control
de LEDs
#ifndef SETUP_H
#define SETUP_H

#include <stdbool.h>
#include "driver/gpio.h"

// Puente H discreto (control de giro del motor CC)
#define PIN_MOTOR_IN1       GPIO_NUM_18
#define PIN_MOTOR_IN2       GPIO_NUM_19

// Sensor ultrasónico PING (señal bidireccional SIG en GPIO 4)
#define PIN_SENSOR_PING     GPIO_NUM_4

// LEDs indicadores de rango de detección
#define PIN_LED_VERDE       GPIO_NUM_21
#define PIN_LED_AMARILLO    GPIO_NUM_22
#define PIN_LED_ROJO        GPIO_NUM_23

// Parámetros geométricos del barrido pendular
#define ANGULO_MINIMO       0
#define ANGULO_MAXIMO       180
#define PASO_ANGULAR        15

// Umbrales de distancia en precisión simple (float 32 bits)
#define UMBRAL_CERCA_CM     15.0f   // Menor a 15 cm  -> Alerta (LED Rojo)
#define UMBRAL_MEDIO_CM     40.0f   // Entre 15 y 40 cm -> Precaución (LED
Amarillo)
#define DISTANCIA_MAX_CM    300.0f  // Rango máximo válido de detección

// Estados de la Máquina de Estados Finitos (FSM)
typedef enum {
    ESTADO_INICIALIZACION = 0,
    ESTADO_BARRIDO_HORARIO,
    ESTADO_LECTURA,
    ESTADO_BARRIDO_ANTIHORARIO
} estado_sistema_t;

void setup_inicializar_hardware(void);
void setup_actualizar_leds(float distancia_cm, bool objeto_detectado);

#endif // SETUP_H
```

4.2. Implementación de Inicialización y Panel de LEDs (setup.c)

Configura mediante una máscara de bits (`pin_bit_mask`) los pines de salida hacia el puente H y los LEDs, deshabilitando las resistencias internas de pull-up y pull-down para no interferir con la etapa externa. La función `setup_actualizar_leds()` evalúa los rangos de distancia mediante estructuras condicionales `if / else if / else` mutuamente excluyentes.

```c
// Archivo: main/setup.c — Configuración de GPIOs y lógica de indicación luminosa
#include "setup.h"

void setup_inicializar_hardware(void)
{
    gpio_config_t io_conf = {
        .pin_bit_mask = (1ULL << PIN_MOTOR_IN1)    |
                        (1ULL << PIN_MOTOR_IN2)    |
                        (1ULL << PIN_LED_VERDE)    |
                        (1ULL << PIN_LED_AMARILLO) |
                        (1ULL << PIN_LED_ROJO),
        .mode = GPIO_MODE_OUTPUT,
        .pull_up_en = GPIO_PULLUP_DISABLE,
        .pull_down_en = GPIO_PULLDOWN_DISABLE,
        .intr_type = GPIO_INTR_DISABLE
    };
    gpio_config(&io_conf);

    // Configurar pin del sensor PING inicialmente como salida en estado bajo
    gpio_reset_pin(PIN_SENSOR_PING);
    gpio_set_direction(PIN_SENSOR_PING, GPIO_MODE_OUTPUT);
    gpio_set_level(PIN_SENSOR_PING, 0);

    // Apagar motor y encender LED verde al iniciar
    gpio_set_level(PIN_MOTOR_IN1, 0);
    gpio_set_level(PIN_MOTOR_IN2, 0);
    gpio_set_level(PIN_LED_VERDE, 1);
    gpio_set_level(PIN_LED_AMARILLO, 0);
    gpio_set_level(PIN_LED_ROJO, 0);
}

void setup_actualizar_leds(float distancia_cm, bool objeto_detectado)
{
    if (!objeto_detectado) {
        // Sin objeto en rango válido: solo LED Verde encendido
        gpio_set_level(PIN_LED_ROJO, 0);
        gpio_set_level(PIN_LED_AMARILLO, 0);
        gpio_set_level(PIN_LED_VERDE, 1);
    } else if (distancia_cm < UMBRAL_CERCA_CM) {
        // Objeto cercano (< 15 cm): Alerta en LED Rojo
        gpio_set_level(PIN_LED_ROJO, 1);
        gpio_set_level(PIN_LED_AMARILLO, 0);
        gpio_set_level(PIN_LED_VERDE, 0);
    } else if (distancia_cm <= UMBRAL_MEDIO_CM) {
        // Objeto a distancia media (15 cm a 40 cm): Precaución en LED Amarillo
        gpio_set_level(PIN_LED_ROJO, 0);
        gpio_set_level(PIN_LED_AMARILLO, 1);
        gpio_set_level(PIN_LED_VERDE, 0);
    } else {
        // Objeto lejano (> 40 cm): LED Verde encendido
        gpio_set_level(PIN_LED_ROJO, 0);
        gpio_set_level(PIN_LED_AMARILLO, 0);
        gpio_set_level(PIN_LED_VERDE, 1);
    }
}
```

4.3. Módulo de Control del Motor CC y Puente H Discreto (motor.h y motor.c)

Encapsula las acciones sobre los pines `GPIO 18` y `GPIO 19`. Antes de invertir el sentido de giro, se fuerza el apagado previo de ambas entradas para prevenir conducciones transitorias cruzadas en las ramas de potencia, aplicando un retardo de asentamiento mecánico tras detener el péndulo.

```
// Archivo: main/motor.h — Tipos de datos del motor y prototipos del Puente H
#ifndef MOTOR_H
#define MOTOR_H

#include <stdint.h>

// Tiempo de activación (ms) para desplazar el péndulo un PASO_ANGULAR (15°)
#define TIEMPO_PASO_MS          120
#define TIEMPO_ASENTAMIENTO_MS  60

typedef enum {
    MOTOR_DETENIDO = 0,        // IN1 = 0, IN2 = 0 (Freno a GND)
    MOTOR_SENTIDO_HORARIO,     // IN1 = 1, IN2 = 0
    MOTOR_SENTIDO_ANTIHORARIO  // IN1 = 0, IN2 = 1
} sentido_motor_t;

void motor_establecer_sentido(sentido_motor_t sentido);
void motor_detener(void);
void motor_avanzar_paso(sentido_motor_t sentido, uint32_t tiempo_ms);

#endif // MOTOR_H
```

```c
// Archivo: main/motor.c — Implementación del driver para el Puente H discreto
#include "motor.h"
#include "setup.h"
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"

void motor_detener(void)
{
    gpio_set_level(PIN_MOTOR_IN1, 0);
    gpio_set_level(PIN_MOTOR_IN2, 0);
}

void motor_establecer_sentido(sentido_motor_t sentido)
{
    // Apagado previo de seguridad para evitar cortocircuito transitorio de rama
    motor_detener();

    switch (sentido) {
        case MOTOR_SENTIDO_HORARIO:
            gpio_set_level(PIN_MOTOR_IN1, 1);
            gpio_set_level(PIN_MOTOR_IN2, 0);
            break;

        case MOTOR_SENTIDO_ANTIHORARIO:
            gpio_set_level(PIN_MOTOR_IN1, 0);
            gpio_set_level(PIN_MOTOR_IN2, 1);
            break;

        case MOTOR_DETENIDO:
        default:
            motor_detener();
            break;
    }
}

void motor_avanzar_paso(sentido_motor_t sentido, uint32_t tiempo_ms)
{
    motor_establecer_sentido(sentido);
    vTaskDelay(pdMS_TO_TICKS(tiempo_ms));
    motor_detener();
    // Pausa de estabilización mecánica antes de disparar el ultrasonido
    vTaskDelay(pdMS_TO_TICKS(TIEMPO_ASENTAMIENTO_MS));
}
```

4.4. Módulo de Adquisición Ultrasónica (sensor.h y sensor.c)

Define la estructura `medicion_radar_t` que vincula la distancia en centímetros con el ángulo del péndulo. La función `sensor_procesar_lectura()` recibe un puntero a la estructura (`medicion_radar_t *medicion`) para modificar directamente los campos en memoria sin duplicar bytes en la pila (stack). El cálculo de distancia se obtiene multiplicando el tiempo de vuelo en microsegundos por la velocidad del sonido (0,0343 cm/µs) y dividiendo por 2 (recorrido de ida y vuelta), equivalente a dividir por 58,3.

```
// Archivo: main/sensor.h — Estructura de medición y prototipos del sensor PING
#ifndef SENSOR_H
#define SENSOR_H

#include <stdbool.h>
#include <stdint.h>

#define TIMEOUT_ECO_US          25000   // 25 ms máximo de espera de eco
#define FACTOR_US_A_CM          58.3f   // Conversión: d(cm) = t(us) / 58.3

typedef struct {
    float distancia_cm;       // Distancia calculada en centímetros
    int16_t angulo_actual;    // Posición angular estimada del péndulo (0° a 180°)
    bool objeto_detectado;    // Bandera de presencia dentro del área de barrido
} medicion_radar_t;

float sensor_medir_distancia_cm(void);
void sensor_procesar_lectura(medicion_radar_t *medicion);

#endif // SENSOR_H
```

```
// Archivo: main/sensor.c — Lectura por ancho de pulso en microsegundos
#include "sensor.h"
#include "setup.h"
#include "esp_timer.h"
#include "rom/ets_sys.h"

float sensor_medir_distancia_cm(void)
{
    // 1. Generar pulso de disparo (Trigger) de 10 microsegundos en
PIN_SENSOR_PING
    gpio_set_direction(PIN_SENSOR_PING, GPIO_MODE_OUTPUT);
    gpio_set_level(PIN_SENSOR_PING, 0);
    ets_delay_us(4);
    gpio_set_level(PIN_SENSOR_PING, 1);
    ets_delay_us(10);
    gpio_set_level(PIN_SENSOR_PING, 0);

    // 2. Cambiar el pin a modo ENTRADA para escuchar el pulso de retorno (Echo)
    gpio_set_direction(PIN_SENSOR_PING, GPIO_MODE_INPUT);

    int64_t t_inicio_espera = esp_timer_get_time();
    while (gpio_get_level(PIN_SENSOR_PING) == 0) {
        if ((esp_timer_get_time() - t_inicio_espera) > TIMEOUT_ECO_US) {
            return -1.0f; // Error por timeout: no se inició el pulso de eco
        }
    }

    // 3. Medir duración en microsegundos del pulso en estado ALTO
    int64_t t_subida = esp_timer_get_time();
    while (gpio_get_level(PIN_SENSOR_PING) == 1) {
        if ((esp_timer_get_time() - t_subida) > TIMEOUT_ECO_US) {
            return -1.0f; // Eco fuera de rango máximo
        }
    }
    int64_t duracion_us = esp_timer_get_time() - t_subida;

    return (float)duracion_us / FACTOR_US_A_CM;
}

void sensor_procesar_lectura(medicion_radar_t *medicion)
{
    if (medicion == NULL) {
        return;
    }

    float d_cm = sensor_medir_distancia_cm();
    medicion->distancia_cm = d_cm;

    if (d_cm > 2.0f && d_cm <= DISTANCIA_MAX_CM) {
        medicion->objeto_detectado = true;
    } else {
        medicion->objeto_detectado = false;
    }
}
```

4.5. Programa Principal y Coordinación de la Máquina de Estados (main.c)

Integra todos los módulos dentro de `app_main()`. En cada ciclo del bucle `while (1)`, ejecuta la rama correspondiente del `switch (estado_actual)`, emite por consola serie el reporte de posición y distancia mediante `ESP_LOGI`, y cede el procesador al planificador de FreeRTOS con `vTaskDelay()`.

```
// Archivo: main/main.c — Máquina de estados principal y coordinación del radar
#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "esp_log.h"

#include "setup.h"
#include "motor.h"
#include "sensor.h"

static const char *TAG = "MAIN_RADAR";

void app_main(void)
{
    estado_sistema_t estado_actual = ESTADO_INICIALIZACION;
    sentido_motor_t direccion_barrido = MOTOR_SENTIDO_HORARIO;

    medicion_radar_t datos_radar = {
        .distancia_cm = 0.0f,
        .angulo_actual = ANGULO_MINIMO,
        .objeto_detectado = false
    };

    while (1) {
        switch (estado_actual) {

            case ESTADO_INICIALIZACION:
                ESP_LOGI(TAG, "Inicializando subsistemas del radar pendular...");
                setup_inicializar_hardware();
                motor_detener();
                datos_radar.angulo_actual = ANGULO_MINIMO;
                direccion_barrido = MOTOR_SENTIDO_HORARIO;
                estado_actual = ESTADO_BARRIDO_HORARIO;
                break;

            case ESTADO_BARRIDO_HORARIO:
                motor_avanzar_paso(MOTOR_SENTIDO_HORARIO, TIEMPO_PASO_MS);
                datos_radar.angulo_actual += PASO_ANGULAR;
                direccion_barrido = MOTOR_SENTIDO_HORARIO;
                estado_actual = ESTADO_LECTURA;
                break;

            case ESTADO_LECTURA:
                sensor_procesar_lectura(&datos_radar);
                setup_actualizar_leds(datos_radar.distancia_cm,
datos_radar.objeto_detectado);

                ESP_LOGI(TAG, "Angulo: %d deg | Distancia: %.2f cm | Detectado:
%d",
                         datos_radar.angulo_actual, datos_radar.distancia_cm,
datos_radar.objeto_detectado);

                if (direccion_barrido == MOTOR_SENTIDO_HORARIO) {
                    estado_actual = (datos_radar.angulo_actual >= ANGULO_MAXIMO)
                                    ? ESTADO_BARRIDO_ANTIHORARIO :
```

5. Guía de Preguntas y Respuestas Clave para la Defensa Individual

- **¿Por qué se pasan punteros (`medicion_radar_t *medicion`) a la función del sensor en lugar de pasar la estructura por valor?** Porque en C los parámetros se pasan por valor (creando una copia temporal). Pasar la dirección de memoria con `&datos_radar` permite que `sensor_procesar_lectura()` modifique directamente los tres campos de la estructura original sin copiar bytes innecesarios en la pila (stack) de la tarea.
- **¿Por qué se emplean dos funciones de retardo distintas (`ets_delay_us` y `vTaskDelay`)?** `ets_delay_us(10)` genera una espera precisa de 10 µs por ciclo de reloj para el pulso de disparo del sensor PING (donde 1 tick de FreeRTOS sería demasiado largo), mientras que `vTaskDelay(pdMS_TO_TICKS(100))` suspende la tarea durante 100 ms cediendo la CPU al sistema operativo para evitar el reinicio del Watchdog Timer.
- **¿Qué ocurre si durante la defensa piden modificar el código en vivo para agregar una zona de alerta crítica a menos de 5 cm que haga parpadear el LED rojo?** Basta con agregar `#define UMBRAL_CRITICO_CM 5.0f` en `setup.h` y añadir un bloque `else if (distancia_cm < UMBRAL_CRITICO_CM)` al inicio de `setup_actualizar_leds()` en `setup.c`, demostrando la ventaja del diseño modular.
- **¿Cómo se verifica con el multímetro que el puente H no tiene cortocircuito de rama y satura correctamente?** Con `IN1 = 3,3 V` e `IN2 = 0 V`, se mide respecto de GND: el nodo de compuertas izquierdo debe marcar ≈ 0,05 V (BS170 saturado e IRF9510 conduciendo), el nodo de compuertas derecho debe marcar +12,0 V (BS170 en corte e IRF510 conduciendo) y entre los terminales del motor deben medirse ≈ 10,8 V.